

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial: Compilador de crochet

Instituto Tecnológico de Buenos Aires



Grupo G127

Integrantes:

- Bertucci, Nicole 64415

Fecha de Entrega: 14/06/2026

Tabla de Contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1 Dominio.....	3
2.2 Lenguaje.....	4
3. Implementación.....	6
3.1 Frontend.....	6
3.2 Backend.....	7
3.3 Dificultades Encontradas.....	8
4. Futuras Extensiones.....	8
5. Conclusiones.....	9
6. Bibliografía.....	9

1. Introducción

En el ámbito del crochet existen dos formas ampliamente utilizadas para representar patrones: mediante descripciones textuales o a través de diagramas compuestos por símbolos estandarizados, generalmente representados en formato SVG. Sin embargo, las descripciones textuales suelen variar según el idioma y carecen de una estandarización formal, lo que dificulta el intercambio y la interpretación de patrones entre distintos usuarios.

El presente trabajo consistió en el desarrollo de un compilador capaz de traducir patrones de crochet escritos en un DSL diseñado específicamente para este proyecto a diagramas SVG equivalentes. El proyecto fue realizado en el marco de la materia Autómatas, Teoría de Lenguajes y Compiladores, con el objetivo de facilitar la creación de diagramas y mejorar su accesibilidad para usuarios principiantes.

El compilador fue desarrollado en lenguaje C. Para la implementación del analizador léxico se utilizó Flex, herramienta encargada de reconocer los distintos componentes léxicos del lenguaje de entrada. Por otro lado, se empleó Bison para la construcción del analizador sintáctico, responsable de verificar que la entrada respete la gramática definida y de generar las estructuras internas necesarias para la creación del diagrama SVG.

El desarrollo del proyecto se dividió en tres etapas. La primera consistió en el diseño de la propuesta, incluyendo la definición de requisitos, la especificación del DSL y la elaboración de ejemplos y casos de prueba. En la segunda etapa se implementó el frontend del compilador, compuesto por los analizadores léxico y sintáctico, cuyo principal objetivo fue la construcción del AST (Árbol de Sintaxis Abstracta) correspondiente a la entrada. Finalmente, en la tercera etapa se completaron las validaciones restantes y se desarrolló la generación automática del diagrama SVG a partir de la representación interna obtenida.

2. Modelo Computacional

El proceso de desarrollo del compilador se basó en tres etapas, como se mencionó anteriormente. En el proceso de diseño, dos aspectos fundamentales que se determinaron fueron el dominio y el lenguaje.

2.1 Dominio

El dominio del compilador desarrollado en este proyecto se centró en la construcción de un diagrama SVG con los símbolos estandarizados para patrones de crochet a partir de un custom DSL que podrán ser utilizadas para compartir patrones más fácilmente entre aficionados. El lenguaje de entrada se diseñó teniendo en cuenta dos tipos de patrones: *round* y *linear*, diferentes tipos de puntos: *mr*, *ch*, *slst*, *sc*, *hdc* y *dc*, y diferentes modificaciones a los puntos: *inc*, *dec*, *flo* y *blo*.

Lamentablemente, no fue posible por el tiempo y la complejidad del proyecto en el crochet incluir todos los puntos y modificaciones del crochet, así que se decidió utilizar los más comunes.

2.2 Lenguaje

El lenguaje de entrada consiste en un DSL diseñado a partir de la notación textual comúnmente utilizada para compartir patrones de crochet. Con el objetivo de generar una grilla adecuada para la representación gráfica, cada patrón incluye una sección *patternInfo*, donde se especifican la forma del patrón, la cantidad de filas y la cantidad máxima de puntos.

A continuación, se define una sección *pattern* que contiene la descripción de cada fila (row) del patrón. Esta sección debe poseer el mismo identificador que su correspondiente *patternInfo*, permitiendo almacenar múltiples patrones en un mismo archivo sin que su información se mezcle.

Dentro de la sección *pattern*, cada fila describe los puntos en orden, indicando su cantidad y, cuándo corresponde, su modificador. Además, es posible definir rangos de filas, por ejemplo row 2-6, para reutilizar una misma descripción en varias filas consecutivas sin necesidad de repetirla. De manera similar, se pueden repetir secuencias de puntos mediante la sintaxis [...] x n, donde n indica la cantidad de repeticiones de la secuencia.

A continuación, se presentan ejemplos de patrones correspondientes a los distintos tipos de figura soportados.

Figura 1: patternInfo del patrón Sock, un patrón de formato *linear*

```
patternInfo Sock{  
  shape: linear;  
  maxWidth: 8;  
  rows: 8;  
}  
  
pattern Sock{  
  row 1: ch 4  
  row 2: sc 4  
  row 3: sc-inc 4  
  row 4-8: sc 8  
}
```

Figura 2: patternInfo del patrón Coaster, un patrón de formato *round*

```

patternInfo Coaster{
  shape: round;
  maxRingSize: 6;
}

pattern Coaster{
  row 1: mr 6
  row 2: sc-inc 6
  row 3: [sc 1 , sc-inc 1] x6
}

```

Para simplificar la escritura de patrones y facilitar su procesamiento durante la compilación, se definió una clave específica en el DSL para cada punto o modificación utilizada en los diagramas de crochet. De esta forma, cada símbolo gráfico posee una representación textual única dentro del lenguaje. La siguiente tabla muestra las claves definidas y los símbolos que representan en el diagrama generado.

Tabla 1 : Claves de los puntos de crochet en el DSL y su símbolo correspondiente

Clave	Símbolo
ch (chain)	
slst (slip stitch)	
mr (magic ring)	
sc (simple crochet)	
hdc (half double crochet)	
dc (double crochet)	

Tabla 2 : Claves de los modificadores de puntos de crochet en el DSL y su símbolo correspondiente*

Clave	Símbolo
inc (increase)	+
dec (decrease)	-
flo (front loop only)	↺
blo (back loop only)	↻

*En el diagrama cada uno de los puntos ocupa un espacio en la grilla, si el punto presenta modificaciones esta se indicará en la parte inferior del punto.

3. Implementación

Para la implementación del compilador se utilizó como base el proyecto [Flex-Bison-Compiler](#) (sobre la rama development), que facilitó el uso de las herramientas Flex (para el análisis léxico) y Bison (para el análisis sintáctico).

3.1 Frontend

Para desarrollar el frontend, el énfasis estuvo principalmente en la definición de una gramática formal que representara el lenguaje de crochet deseado. De esta manera se definieron como símbolos terminales: enteros (para cantidades de puntos y rangos de filas), identificadores (para los nombres de patrones), y diversos tipos de tokens que representan palabras reservadas del lenguaje. Por otro lado, se definieron como no terminales las estructuras de más alto nivel:

- Los bloques *patternInfo* y *pattern*, que juntos conforman la definición completa de un patrón.
- Las propiedades de configuración del bloque *patternInfo* (*shape*, *maxWidth*, *maxRingSize*, *rows*) y los tipos de forma (*linear*, *round*).
- Las declaraciones de filas (*rowDecl*), tanto simples (*row N:*) como rangos (*row N-M:*), y las listas de instrucciones que las componen.
- Los ítems de puntada (*stitchItem*), que pueden ser una puntada simple con modificador y cantidad opcionales, o un bloque de repetición (*[...] x N*).
- El símbolo inicial, definido como *program*, del cual se deriva la estructura completa del patrón de crochet.

Además, se intentaron modularizar las acciones sintácticas de Bison agrupando conceptos relacionados. Con este objetivo se crearon estructuras generales como *StitchList*, que representa una secuencia ordenada de instrucciones dentro de una fila, *RowList*, que agrupa todas las filas de un patrón, e *InfoPropList*, que contiene las propiedades de configuración de un

bloque *patternInfo*. De manera similar, en el lexer se agruparon las acciones de Flex por categoría semántica (*StitchLexemeAction*, *ModifierLexemeAction*, *KeywordLexemeAction*, entre otras), reduciendo la duplicación y facilitando la lectura del scanner.

Desde el frontend fue posible validar numerosas estructuras y descartar aquellas consideradas inválidas dentro del lenguaje. Entre las validaciones que realiza la gramática se encuentran: el uso exclusivo de tipos de puntada reconocidos (*ch*, *sc*, *hdc*, *dc*, *slst*, *mr*), la correcta aplicación de modificadores (*-inc*, *-dec*, *-blo*, *-flo*) como sufijos de una puntada, la sintaxis del bloque de repetición ($[\dots] \times N$), y la obligatoriedad de que cada patrón posea tanto un bloque *patternInfo* como un bloque *pattern* con el mismo nombre. Sin embargo, hubo casos que, debido a la naturaleza de la gramática libre de contexto, debieron postergarse para validaciones semánticas del backend: la verificación de que el nombre del bloque *patternInfo* coincida con el del bloque *pattern*, la consistencia entre el tipo de forma declarado y las propiedades presentes (por ejemplo, que un patrón *Round* declare *maxRingSize* y no *maxWidth*), el orden secuencial correcto de los números de fila, y la verificación de que la primera fila de un patrón *Round* contenga exclusivamente un *mr*.

3.2 Backend

La primera etapa del desarrollo backend del compilador consistió en el análisis del lenguaje de crochet elaborado previamente, con el objetivo de determinar las validaciones semánticas necesarias sobre el AST. A diferencia de lenguajes con variables o funciones, el dominio del crochet no requirió una tabla de símbolos tradicional, ya que la información relevante se encuentra completamente estructurada en el árbol: los patrones tienen nombre, propiedades de configuración y filas de instrucciones. Sin embargo, sí fue necesario validar la unicidad de los nombres de patrones a lo largo del programa, verificando que no existieran dos definiciones con el mismo identificador.

Las validaciones semánticas implementadas se organizaron en dos niveles. El primero abarca validaciones estructurales por patrón: que el bloque *patternInfo* y el bloque *pattern* compartan el mismo nombre, que cada tipo de forma (*linear* o *round*) tenga exactamente las propiedades requeridas sin duplicados ni propiedades ajenas, que los números de fila sean secuenciales comenzando desde 1 sin saltos ni repeticiones, y que los rangos de filas sean válidos. El segundo nivel abarca validaciones de contenido: que los counts de stitches y repeticiones sean positivos, que los modificadores no se apliquen cuando no hay suficientes puntos disponibles en la fila anterior, y que los patrones *Round* comiencen obligatoriamente con *mr* como primer stitch de la primera fila.

Adicionalmente, se implementó una validación de continuidad entre filas, que resultó ser la más compleja del análisis semántico. Cada fila consume una cantidad determinada de puntos de la fila anterior: los stitches normales, *-inc*, *-blo* y *-flo* consumen un punto cada uno, mientras que *-dec* consume dos puntos por cada aplicación, ya que trabaja sobre dos puntos de la fila anterior simultáneamente. Los stitches *ch* y *mr* no consumen puntos de ninguna fila anterior. A partir de esto, se valida que ninguna fila intente consumir más puntos de los que la fila anterior produce, y que la primera fila no contenga stitches que requieran una fila previa (solo pueden contener *mr* o *ch*).

Para la generación de código, el compilador produce un archivo SVG con el nombre del archivo de entrada, escribiendo directamente a un archivo con su mismo nombre en lugar de a la salida estándar (para esto hubo que realizar cambios dentro del archivo [run.sh](#)). La representación visual de cada patrón depende de su tipo: los patrones *linear* se renderizan como una grilla rectangular donde cada celda representa un stitch, mientras que los patrones *round* se renderizan como anillos concéntricos donde los stitches se distribuyen uniformemente a lo largo de cada anillo mediante coordenadas polares. Cada tipo de stitch tiene su propio símbolo SVG siguiendo la notación estándar del crochet.

3.3 Dificultades Encontradas

Un desafío particular surgió en la representación visual de los modificadores *-inc* y *-dec*. En un primer enfoque se utilizó una grilla uniforme donde cada stitch ocupaba la misma celda, pero esto resultaba visualmente incorrecto: un *sc-inc 4* en una fila y *sc 8* en la siguiente no quedaban alineados, ya que el *increase* produce dos puntos por cada stitch ejecutado. La solución adoptada fue asignar a cada stitch con *-inc* o *-dec* el doble de espacio horizontal en la grilla, centrando el símbolo entre las dos columnas que ocupa. Esto permite que las filas queden visualmente alineadas, reflejando correctamente la relación entre los puntos consumidos y producidos en cada fila. Para simplificar su representación y facilitar la generación automática, se decidió incorporar los símbolos “+” y “-” para indicar explícitamente estas modificaciones. Si bien esta solución no reproduce exactamente la notación tradicional, permite representar estas operaciones de manera clara y consistente.

4. Futuras Extensiones

Si bien el lenguaje desarrollado cumple con los objetivos planteados inicialmente, existen diversas posibilidades de expansión. Una de las más interesantes consiste en incorporar soporte para una mayor variedad de puntos y modificaciones utilizadas en patrones de crochet, permitiendo representar diagramas más complejos y cubrir un conjunto más amplio de patrones reales.

Asimismo, sería deseable mejorar la generación de los diagramas SVG para que la disposición de los puntos se asemeje aún más a la utilizada en los diagramas de crochet tradicionales. En particular, podrían explorarse nuevas estrategias de representación para operaciones como *increase* y *decrease*, con el objetivo de obtener diagramas visualmente más fieles a las convenciones empleadas por la comunidad de crochet sin perder la capacidad de generarlos de manera automática.

Por último, una posible extensión del DSL sería incorporar mecanismos de reutilización entre patrones definidos en un mismo archivo. Esto permitiría referenciar y reutilizar filas o secuencias pertenecientes a otros patrones, reduciendo la duplicación de código y facilitando la definición de patrones complejos contruidos a partir de componentes comunes.

5. Conclusiones

Como conclusión del proyecto, se pudo llevar a cabo el desarrollo de un compilador completo, pasando por las tres etapas propuestas por la cátedra: comenzando con el diseño del lenguaje y continuando con el desarrollo del frontend y el backend.

A su vez, el proyecto permitió una visualización más concreta de las temáticas estudiadas en la materia, pudiendo elegir el dominio y desarrollar de forma integral los análisis léxico, sintáctico y semántico, así como la generación de código. El dominio del *crochet* resultó ser una elección interesante, ya que si bien las construcciones del lenguaje son relativamente acotadas, las validaciones semánticas entre filas —en particular la continuidad de puntos entre una fila y la siguiente considerando los efectos de *-inc* y *-dec*— representaron un desafío conceptual no trivial. El proyecto base provisto por la cátedra fue una herramienta fundamental para el cumplimiento de la tarea asignada, aunque representó un nuevo desafío al basarse en tecnologías con las que no se había trabajado previamente, como Flex y Bison.

6. Bibliografía

1.  (2026-03-12) Proyecto Especial
2.  (2026-03-12) Modelo Computacional
3.  (2026-03-08) Especificación
4.  (2025-09-03, v1.0.0) Análisis Léxico
5.  (2024-05-08, v0.1.0) Análisis Sintáctico
6.  (2024-09-25, v0.2.0) Análisis Semántico
7.  (2024-05-16, v0.1.0) Tabla de Símbolos
8.  (2024-05-16, v0.1.0) Scopes
9.  (2024-05-16, v0.1.0) Sistema de Tipos
10.  (2024-05-16, v0.1.0) Generación de Código
11.  (2024-05-16, v0.1.0) Runtime